HearthGen for GraphML: LLM-Augmented Semantic Knowledge Graphs for Hearthstone Custom-Card Retrieval

Yicheng Jin **Zhiheng Wang** **Zicong Zhang**
Rice University
https://github.com/TraceOnSnow/HearthstoneCardGenerator

Abstract

Hearthstone custom-card design is a community-driven creative task in which users describe new cards through class identity, mechanics, stat lines, generated tokens, or references to existing cards. This makes the problem naturally graph-structured: cards connect to classes, types, keywords, actions, targets, resources, and derived-card relations. We present HearthGen as a GraphML project: an LLM-augmented semantic knowledge graph for retrieving mechanically relevant Hearthstone cards from natural-language custom-card requests. The system converts official card data into structured semantics, constructs a bipartite-style card–attribute graph, parses user requests into graph queries, and compares graph-based retrieval against TF-IDF, CLIP, and Node2Vec baselines. We also evaluate graph representation learning through held-out edge prediction. On 19 community-style DIY prompts, interpretable semantic KG retrieval achieves the best overall relevance@5 score (0.806), outperforming TF-IDF (0.679), CLIP (0.456), and Node2Vec graph embeddings (0.700). Node2Vec still learns useful graph structure, achieving 0.839 ROC-AUC and 0.858 average precision on held-out card-attribute edge prediction. These results show that Hearthstone card retrieval benefits from explicit semantic graph structure, while learned graph embeddings provide a complementary GraphML baseline.

Keywords: knowledge graphs, graph machine learning, retrieval-augmented generation, large language models, Node2Vec, game content generation

1 Introduction

Hearthstone custom-card design is popular in online communities where players create new cards, discuss balance, and generate artwork for their ideas. A user may provide a vague request such as "make a defensive Warrior card," a mechanic-first request such as "a spell that freezes enemy minions," or a nearly complete card text that only needs artwork. The task is not simply text-to-image generation. A useful system must understand class identity, card type, mechanics, keywords, generated tokens, and references to existing card families.

This project studies the problem as a graph machine learning and LLM-graph synergy task. Hearthstone cards are naturally graph-structured: each card connects to semantic attributes such as class, type, keyword, action, target, resource, spell school, generated cards, and related cards. Instead of treating a card as unstructured text, we build a semantic knowledge graph (KG) where card nodes connect to typed attribute nodes. Natural-language custom-card requests are parsed into the same attribute space, allowing graph retrieval to match gameplay semantics.

Preprint. Under review.

Figure 1: **End-to-end HearthGen case study.** A vague community-style request is parsed into graph attributes, matched against faceted KG evidence, converted into a playable card design and LoRA caption, and rendered as Hearthstone-style artwork. For the 559 project, the central technical object is the semantic graph and retrieval over it; image generation demonstrates the complete application.

A motivating example is the Hearthstone "Rager" meme. Magma Rager is a classic 3-mana 5/1 minion, and later cards with names such as Ice Rager and Faceless Rager continued the high-attack, low-health joke. If a user asks for "a defensive Control Warrior Rager," the system must combine several signals: the Rager family reference, the 5/1 statline convention, and Warrior armor mechanics. Pure text retrieval can overfocus on keywords, and CLIP retrieval can return visually plausible but mechanically irrelevant images. A semantic graph can explicitly connect the request to both Rager-family anchors and Warrior armor cards.

Our contributions are:

- We construct a semantic Hearthstone KG from all card records, including collectible and derived cards.
- We combine rule-based extraction and LLM enrichment to create structured card semantics and natural-language query parses.
- We evaluate four retrieval methods: TF-IDF, CLIP, interpretable semantic KG matching, and Node2Vec graph embedding retrieval.
- We evaluate GraphML structure learning through held-out card-attribute edge prediction.
- We show a downstream application in KG-augmented card design and artwork generation.

2 Related Work

Graph representation learning. Random-walk graph embeddings such as DeepWalk and Node2Vec learn node representations by treating sampled graph walks as sentences [Perozzi et al.,

2014, Grover and Leskovec, 2016]. Graph neural networks such as GCN and GraphSAGE instead propagate features over graph neighborhoods [Kipf and Welling, 2017, Hamilton et al., 2017]. Our graph is heterogeneous and close to bipartite: cards connect to typed semantic attributes. We use Node2Vec because it is simple, scalable, and appropriate as a baseline for whether learned graph embeddings capture useful card-attribute structure.

Retrieval-augmented generation. Retrieval-augmented generation conditions downstream generation on external evidence [Lewis et al., 2020]. Here, retrieval evidence is not free-form documents but structured card nodes and artwork references. This makes the retrieval stage interpretable: a result can be justified by shared class, type, action, keyword, or related-card nodes.

Multimodal baselines. CLIP maps text and images into a shared embedding space and is commonly used for text-to-image retrieval [Radford et al., 2021]. CLIP is a strong visual baseline, but it does not explicitly encode gameplay mechanics. Our experiments compare CLIP retrieval with graph-based retrieval to test whether semantic graph structure adds value beyond visual similarity.

Game content and image generation. We use Stable Diffusion as a downstream image generator [Rombach et al., 2022] and a LoRA adapter for Hearthstone style adaptation [Hu et al., 2021]. These components are not the main GraphML contribution; they demonstrate that retrieved graph evidence can support a realistic creative pipeline.

3 Data and Semantic Graph Construction

3.1 Card Data

We use Hearthstone card data from HearthstoneJSON and HearthSim tooling [Zero-to-Heroes, 2026, HearthSim, 2026]. The dataset contains 8,661 total cards after filtering Battlegrounds and special-mode cards from the main semantic pipeline, with 6,149 collectible cards that also have artwork references for retrieval and generation experiments. Each raw card includes metadata such as class ID, card type ID, rarity, mana cost, attack, health, card text, keywords, and child card IDs.

3.2 Structured Semantics

We first convert raw card records into structured semantic records. Deterministic metadata fields are mapped directly from IDs to names: class, card type, set, rarity, spell school, minion type, mana cost, attack, health, durability, and keywords. Rule-based extractors identify common gameplay actions from card text, including `deal_damage`, `summon`, `draw`, `gain_armor`, `heal`, `freeze`, and `destroy`. LLM enrichment using MiniMax-M2.7 is used only for fields that rules often miss, such as implicit related-card references, generated-card roles, and concise semantic summaries [MiniMax, 2026]. This design keeps factual fields deterministic while allowing LLMs to enrich ambiguous card semantics.

3.3 Semantic Knowledge Graph

The semantic KG is close to a bipartite graph. One side contains card nodes; the other side contains typed semantic attribute nodes. Attribute node types include class, card type, keyword, action, target, resource, spell school, minion type, generated card name, related card name, trigger, and condition. Edges include `HAS_CLASS`, `HAS_CARD_TYPE`, `HAS_KEYWORD`, `PERFORMS_ACTION`, `TARGETS`, `AFFECTS_RESOURCE`, `GENERATES_CARD`, and `HAS_CHILD_CARD`.

Formally, we write the graph as

$$G = (V_c \cup V_a, E), \tag{1}$$

where V_c is the set of card nodes and V_a is the set of semantic attribute nodes. Each typed edge is represented as (c, a, r), where $c \in V_c$, $a \in V_a$, and r is the relation type. This representation is intentionally not a generic text graph: relation types preserve gameplay meaning, so matching `PERFORMS_ACTION: gain_armor` is different from matching a visually similar card or a nearby word.

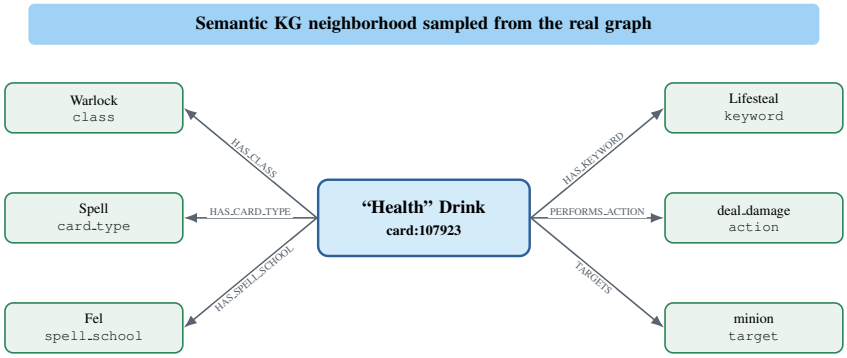


Figure 2: **Semantic KG neighborhood sampled from the real graph.** The card node "Health" Drink connects to typed semantic attributes such as class, card type, spell school, keyword, action, and target. User queries are mapped into the same attribute space, allowing graph retrieval to match gameplay semantics.

4 Methods

4.1 Natural-Language Query Parsing

Each user request is converted into a structured query over KG attribute types. The query contains fields such as class, card type, actions, keywords, spell schools, generated card names, and related card names. We use deterministic pattern matching for common terms and LLM parsing when richer interpretation is needed. For example, "defensive Control Warrior Rager" maps to class `Warrior`, type `Minion`, action `gain_armor`, and related card names `Rager` and `Magma Rager`.

The parsed query is a set of typed attribute constraints $A_q = \{(a, t)\}$ plus a short generated card description. This mirrors the examples supplied for the course: the system must define the graph object, the prediction/retrieval task, and the representation used by the graph model rather than only describing a software pipeline.

4.2 Retrieval Methods

We compare four retrieval methods.

TF-IDF baseline. We retrieve cards by lexical similarity over generated LoRA captions and semantic summaries. This baseline captures literal keyword overlap but does not use graph structure.

CLIP baseline. We retrieve card artworks by CLIP text-to-image similarity. This baseline tests whether visual embedding similarity alone can find useful references.

Semantic KG matching. We map query fields to KG attribute nodes and score each card by weighted neighborhood overlap. Actions and keywords receive higher weight than weak text overlap. This method is interpretable because each retrieved card includes matching graph reasons.

Node2Vec graph embedding retrieval. We train Node2Vec on the semantic KG. A query embedding is computed by averaging the embeddings of its matched attribute nodes, and cards are ranked by cosine similarity to this query embedding. This is the main learned GraphML retrieval baseline.

For explicit KG matching, the score is

$$s_{\mathrm{KG}}(c, q) = \sum_{a \in N(c) \cap A_q} w_{\tau(a)} + \lambda \cdot \mathrm{textOverlap}(c, q), \tag{2}$$

where $\tau(a)$ is the attribute type and $w_{\tau(a)}$ gives higher weight to actions, keywords, class, and related-card nodes. For Node2Vec retrieval, let z_v be the learned embedding of node v. We embed

Method	Class@5	Action@5	Relation@5	Overall@5
TF-IDF	0.7105	0.5337	**0.7033**	0.6788
CLIP	0.4463	0.1258	0.5421	0.4557
Node2Vec	0.7199	0.5535	0.6582	0.7002
Semantic KG	**0.8105**	**0.8145**	0.6188	**0.8061**

Table 1: **Retrieval relevance on 19 DIY prompts.** Semantic KG matching performs best overall because it directly matches typed gameplay attributes. Node2Vec improves over TF-IDF overall but is less precise than explicit KG matching for action semantics.

a query by averaging matched attribute embeddings:

$$z_q = \frac{1}{|A_q|} \sum_{a \in A_q} z_a, \qquad s_{\mathrm{N2V}}(c, q) = \cos(z_c, z_q). \tag{3}$$

This comparison separates two graph-based choices: symbolic typed-neighborhood matching versus learned random-walk embeddings.

4.3 Held-Out Edge Prediction

To test whether the graph representation captures reusable structure, we also run a link prediction experiment. We sample card-attribute edges, hold out 15%, train Node2Vec on the remaining graph, and train a logistic regression classifier over edge features. Edge features concatenate the elementwise product and absolute difference of endpoint embeddings. We evaluate with ROC-AUC and average precision on balanced positive and negative test edges.

The link-prediction feature for a candidate edge (u, v) is

$$\phi(u, v) = [z_u \odot z_v \,;\, |z_u - z_v|], \tag{4}$$

and the classifier estimates $p((u, v) \in E) = \sigma(\theta^\top \phi(u, v))$. This task evaluates whether graph embeddings can recover missing card–attribute facts, independently from the LLM query parser.

5 Experiments

5.1 DIY Retrieval Evaluation

We evaluate retrieval on 19 natural-language DIY prompts designed to reflect realistic custom-card use cases. The prompts include vague creative requests, mechanic-first requests, implicit related-card references, fully specified card text, and art-first requests. Each retrieval method returns top-5 cards. We evaluate class match, action match, relation match, and overall relevance using an LLM judge, then inspect qualitative cases manually.

For the GraphML run, Node2Vec uses 64 dimensions, walk length 12, 8 walks per node, window size 5, and random seed 13. The graph used for Node2Vec contains 6,149 card nodes and 68,223 typed card-attribute edges before filtering to the undirected embedding graph.

5.2 GraphML Edge Prediction

The edge prediction task evaluates learned graph structure independently from the final retrieval pipeline. We sample 3,000 card-attribute edges, hold out 15%, train Node2Vec on the remaining graph, and evaluate a logistic classifier on 900 balanced test edges.

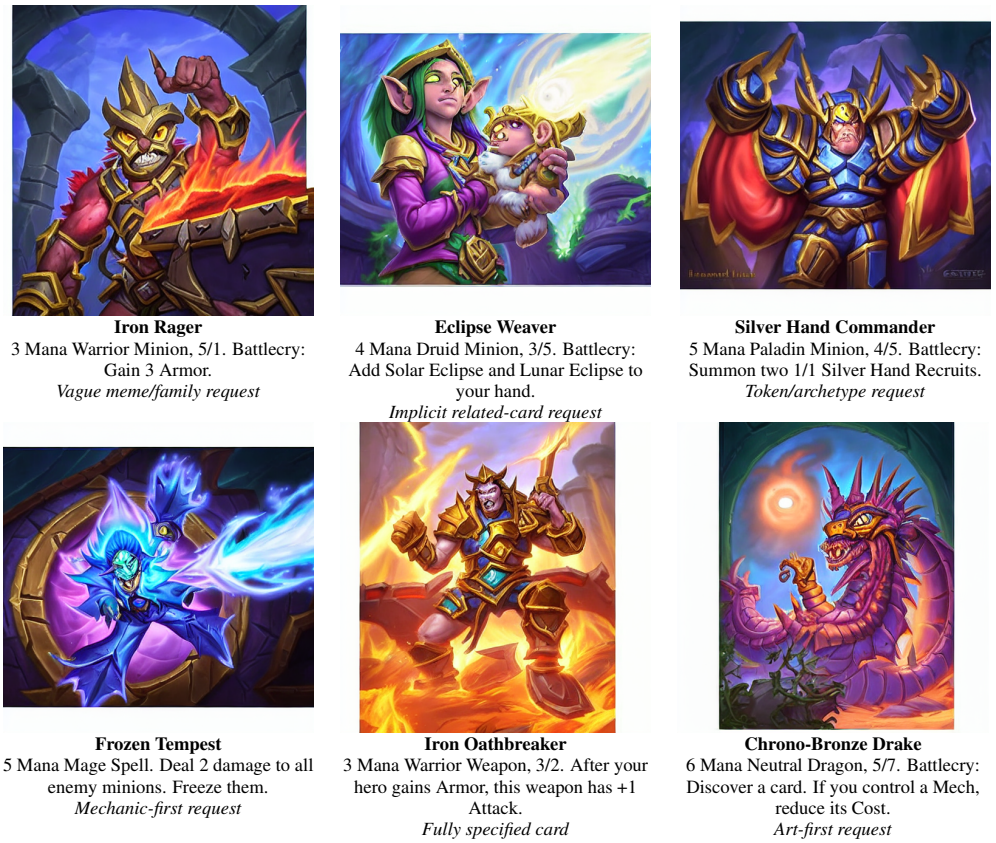
Negative examples are sampled from card–attribute pairs that do not appear as KG edges. We keep the negative set balanced with positives so ROC-AUC and average precision measure ranking quality rather than majority-class behavior.

5.3 Downstream Generation Evaluation

Although the main 559 contribution is graph-based retrieval, we also evaluate and visualize the downstream image-generation application. We compare base Stable Diffusion 1.5 versus a

Method	Train edges	Test edges	ROC-AUC	AP
Node2Vec + logistic regression	5100	900	0.8389	0.8579

Table 2: **Held-out card-attribute edge prediction.** Node2Vec embeddings capture useful semantic graph structure, achieving strong ROC-AUC and average precision on unseen edges.



Iron Rager
3 Mana Warrior Minion, 5/1. Battlecry: Gain 3 Armor.
Vague meme/family request

Eclipse Weaver
4 Mana Druid Minion, 3/5. Battlecry: Add Solar Eclipse and Lunar Eclipse to your hand.
Implicit related-card request

Silver Hand Commander
5 Mana Paladin Minion, 4/5. Battlecry: Summon two 1/1 Silver Hand Recruits.
Token/archetype request

Frozen Tempest
5 Mana Mage Spell. Deal 2 damage to all enemy minions. Freeze them.
Mechanic-first request

Iron Oathbreaker
3 Mana Warrior Weapon, 3/2. After your hero gains Armor, this weapon has +1 Attack.
Fully specified card

Chrono-Bronze Drake
6 Mana Neutral Dragon, 5/7. Battlecry: Discover a card. If you control a Mech, reduce its Cost.
Art-first request

Figure 3: **LoRA + KG-reference qualitative outputs.** Six representative user prompts are shown as final generated artwork with the corresponding card description. The set mixes vague creative requests, implicit Hearthstone references, mechanic-first briefs, fully specified card text, and art-first requests.

Hearthstone LoRA adapter, and text-only versus reference-conditioned generation. The reference-conditioned setting uses retrieved Hearthstone artwork as visual evidence. Figure 3 shows the final LoRA + KG-reference outputs for six representative prompts.

6 Results and Discussion

Table 1 shows that explicit semantic KG retrieval performs best overall. The largest gain is in action matching: 0.8145 for KG retrieval versus 0.5535 for Node2Vec, 0.5337 for TF-IDF, and 0.1258 for CLIP. This supports the main hypothesis of the project: Hearthstone custom-card requests are structured around gameplay semantics, and a typed card-attribute graph represents those semantics better than raw text or visual similarity alone.

Node2Vec is still useful. It improves overall retrieval over TF-IDF and CLIP, and Table 2 shows that learned graph embeddings capture card-attribute structure well. However, learned embeddings blur some exact semantics. For example, graph proximity may retrieve cards in the same class or archetype but miss the specific requested action. In contrast, explicit KG matching can preserve hard constraints such as `PERFORMS_ACTION = gain_armor`. This is a useful finding for GraphML

Method	Prompt Align.	HS Style	Ref. Sim.	Quality
SD text-only	0.2958	0.7499	–	0.4972
SD + reference	0.2951	0.8366	0.7962	0.5024
LoRA text-only	**0.3013**	0.8581	–	**0.5022**
LoRA + reference	0.2968	**0.8608**	**0.8303**	0.5008

Table 3: **Downstream generation proxy metrics.** LoRA improves Hearthstone style, and reference conditioning increases similarity to retrieved card evidence. These image metrics are secondary to the GraphML evaluation.

applications: learned graph embeddings are strong for structural generalization, while symbolic graph matching remains valuable when attributes are semantically typed and sparse.

CLIP performs worst for gameplay relevance. This is expected because CLIP sees visual and textual similarity, not Hearthstone rule structure. It can retrieve fantasy images that look plausible but fail to match class or mechanic. TF-IDF does better than CLIP for text-heavy prompts but over-relies on literal keywords and can miss implicit card relationships.

The Rager case illustrates why graph evidence is useful. A flat ranking may over-rank Warrior armor cards because they match class and action, while under-ranking Magma Rager because it does not match armor mechanics. For card design, both are needed: Magma Rager provides the family/statline anchor, while Warrior armor cards provide mechanic evidence. We therefore view KG retrieval not only as a top-k ranking problem but also as evidence construction over multiple semantic facets.

The downstream generation results in Table 3 are consistent with the retrieval story. LoRA improves style similarity, while reference conditioning makes the generated image closer to retrieved evidence. However, automatic image metrics are only proxy measures. The main technical contribution for this course is the graph representation and retrieval method, not the image model itself.

7 Challenges and Limitations

The first challenge was semantic extraction. Hearthstone card text contains implicit mechanics, generated tokens, and references to other cards. Pure rules cover common actions such as damage, healing, armor, and summoning, but LLM enrichment is needed for implicit relationships such as Solar Eclipse/Lunar Eclipse or Rager-family references.

The second challenge was evaluation. There is no standard benchmark for Hearthstone custom-card retrieval. We created 19 community-style prompts and used an LLM judge to score retrieval relevance. This is practical for a course project, but future work should include larger human evaluation.

The third challenge was balancing interpretability and learned GraphML. Node2Vec provides a clean graph embedding baseline and performs well on link prediction, but explicit KG matching is more interpretable and more accurate for exact mechanic retrieval. A stronger future system could combine both, for example by using KG rules for hard constraints and graph embeddings for soft semantic expansion.

8 Student Contributions

All team members contributed to project design and final analysis. Yicheng Jin focused on semantic representation design, KG construction, retrieval evaluation, report writing, and integration. Zhiheng Wang contributed to data processing, LoRA/diffusion experiments, generation evaluation, and final result preparation. Zicong Zhang contributed to baseline retrieval, experiment scripting, qualitative examples, and evaluation support.

9 Conclusion

HearthGen demonstrates a novel LLM-graph application for game-content generation. By converting Hearthstone cards into a semantic card-attribute graph, the system can retrieve mechanically

relevant and interpretable evidence from natural-language custom-card requests. Explicit KG retrieval outperforms TF-IDF, CLIP, and Node2Vec retrieval on overall relevance, while Node2Vec achieves strong held-out edge prediction performance. These results suggest that structured game domains are a promising setting for combining LLMs, knowledge graphs, and graph representation learning.

References

Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017.

HearthSim. Hearthsim python hearthstone data. `https://github.com/HearthSim/python-hearthstone-data`, 2026. Accessed May 2026.

Edward J. Hu, Yelong Shen, Phillip Wallis, Yuanzhi Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

MiniMax. Minimax m2.7 text generation api. `https://platform.minimaxi.com/docs`, 2026. Accessed May 2026.

Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Bjorn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

Zero-to-Heroes. Hearthstonejson. `https://github.com/Zero-to-Heroes/HearthstoneJSON`, 2026. Accessed May 2026.